

Task 1 — Web App QA & Debug Report

Field	Details
App Tested	RealWorld "Conduit" — Angular Frontend
Live Demo URL	https://demo.realworld.show/
Source Repo	https://github.com/realworld-apps/angular-realworld-example-app
Date Tested	June 2, 2026
Tester	Manan Sharma

Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	Missing Security Headers — No CSP, X-Frame-Options, X-Content-Type-Options, HSTS	1. Open browser DevTools → Network tab. 2. Load <code>https://demo.realworld.show/</code> . 3. Inspect response headers. Or run: <code>curl -sI https://demo.realworld.show/</code>	Expected: Response includes <code>Content-Security-Policy</code> , <code>X-Frame-Options: DENY</code> , <code>X-Content-Type-Options: nosniff</code> , and <code>Strict-Transport-Security</code> headers. Actual: None	Critical	Nginx server config was not hardened. No <code>add_header</code> directives for security headers. <code>server_tokens off</code> was not set either.

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
			of these headers are present. Only <code>server: nginx/1.29.1</code> is returned, which also leaks the server version.		
2	Page <code><title></code> is always "Conduit" — never updates on navigation	1. Navigate to Home (/) — note browser tab title. 2. Navigate to <code>/login</code> — note title. 3. Navigate to <code>/article/how-to-learn-javascript-efficiently</code> — note title. 4. Navigate to <code>/register</code> — note title. All say "Conduit".	Expected: Title dynamically updates per route, e.g. "Sign In — Conduit", "How to Learn JavaScript — Conduit". This is critical for SEO and screen reader accessibility. Actual: Title is always the static string <code>Conduit</code> on every single route.	Medium	Angular app does not use the <code>Title</code> service to set <code>document.title</code> on route changes. The <code><title></code> tag in <code>index.html</code> is hardcoded and never modified at runtime.
3	No Open Graph / meta description tags — social	1. Copy any article URL (e.g. <code>https://demo.realworld.io</code>)	Expected: Each page should include <code><meta</code>	Medium	Inherent limitation of client-side rendering (CSR).

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
	sharing is completely broken	ld.show/article/how-to-learn-javascript-efficiently). 2. Paste in Slack, Discord, Twitter, or Facebook. 3. Observe the link preview — it's blank/generic. 4. Verify: <code>curl -s https://demo.realworld.show/ grep -i "og:description"</code> returns nothing.	name="description">, og:title, og:description, og:image, twitter:card tags. Actual: Zero meta/OG tags present. Link previews show nothing useful.		Server sends the same static <code>index.html</code> shell for all routes. Without SSR or pre-rendering, social crawlers see no dynamic content.
4	Tags cannot be added or removed when editing an existing article	1. Log in to the app. 2. Create a new article with tags (e.g. "javascript", "tutorial"). 3. Open the article, click "Edit Article". 4. Try to add a new tag in the tag input field. 5. Try to	Expected: Tags should be fully editable — add new tags, remove existing ones, and changes persist on save. Actual: Tag modifications are silently ignored.	High	The editor component does not correctly bind the modified tag array to the PUT request payload. The <code>tagList</code> field is either missing from the request body or the

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
		remove an existing tag. 6. Click "Publish Article". 7. Observe that tags remain unchanged.	Article retains original tags. Confirmed open bug: GitHub Issue #343 .		component doesn't track tag state changes.
5	Ionicons loaded via outdated, protocol-relative external CDN (v2.0.1) without SRI	1. View page source of any page. 2. Find: <code>href="//code.ionicframework.com/ionicons/2.0.1/css/ionicons.min.css"</code> . 3. Note: (a) version is 2.0.1 (current is 7.x), (b) protocol-relative URL <code>//</code> is deprecated, (c) no <code>integrity</code> attribute, (d) external CDN = single point of failure.	Expected: Icon library should be current version, loaded over explicit HTTPS, ideally self-hosted or with Subresource Integrity (SRI) hashes. Actual: Ancient v2.0.1 from external CDN. If CDN goes down, all icons break. CDN compromise could inject malicious code.	Medium	Legacy dependency from the original RealWorld template that was never updated. The icon classes (<code>ion-compose</code> , <code>ion-gear-a</code>) are Ionicons v2 API.
6	No client-side input validation	1. Navigate to <code>/register</code> . 2. Leave all	Expected: Client-side validation prevents	Medium	Registration component relies entirely on

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
	on registration form	fields empty. 3. Click "Sign up" — form submits. 4. Enter "notanemail" as email and "1" as password. 5. Click "Sign up" — form submits again.	empty/invalid submissions. Inline errors like "Email is required", "Password must be at least 8 characters". Email format validated before API call. Actual: Form submits to API with empty/invalid data. Errors only come from server, causing unnecessary network requests and poor UX. No password strength requirements shown.		server-side validation. Does not implement Angular reactive form validators (<code>Validators.required</code> , <code>Validators.email</code> , <code>Validators.minLength</code>).
7	Non-existent user profile shows blank page — no error feedback	1. Navigate to <code>https://demo.realworld.show/profile/nonexistentuser12345</code> . 2. Observe the page.	Expected: A clear "User not found" message or redirect to a 404 page when API returns an error. Actual: Page loads	Low	Profile component does not handle the API 404 error. It silently catches/ignores the error and renders

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
			with profile layout but completely empty — no username, no bio, no articles, no indication the user doesn't exist. User is left confused.		the template with undefined data, resulting in blank fields.

Root-Cause Analysis — Issue #1: Missing Security Headers

What is happening: When any page of the Conduit application is loaded, the server responds with HTTP headers that contain no security directives. Running `curl -sI https://demo.realworld.show/` reveals only basic headers: `server: nginx/1.29.1`, `content-type: text/html`, `cache-control: no-cache`, and standard date/length headers. There is no `Content-Security-Policy`, no `X-Frame-Options`, no `X-Content-Type-Options`, no `Strict-Transport-Security`, and no `Referrer-Policy`. The server also actively discloses its exact version (`nginx/1.29.1`).

Why this happens: The Nginx configuration file was set up to serve the Angular SPA's static files but was never hardened with security headers. This is a classic oversight in AI-assisted "vibe-coded" deployments where the priority is getting the app running rather than making it production-ready. Since the Angular app is a pure client-side SPA with no server middleware, security headers must be added at the Nginx reverse proxy level — there's no application-level code that can inject them.

Why it matters: Without `X-Frame-Options: DENY`, the app can be embedded in iframes on malicious sites for clickjacking attacks. Without `Content-Security-Policy`, there is no browser-enforced restriction on what scripts, styles, or resources can load — amplifying any XSS vulnerability. Without `Strict-Transport-Security`, users visiting via HTTP are vulnerable to SSL-stripping man-in-the-middle attacks. The server version leak gives attackers a specific target to search for known CVEs.

How to fix it: Add the following block to the Nginx `server` configuration:

```
server {  
  
    # Hide server version  
  
    server_tokens off;  
  
    # Security headers  
  
    add_header X-Frame-Options "DENY" always;  
  
    add_header X-Content-Type-Options "nosniff" always;  
  
    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains" always;  
  
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;  
  
    add_header Content-Security-Policy "default-src 'self'; script-src 'self'; style-src 'self'  
'unsafe-inline' https://fonts.googleapis.com https://code.ionicframework.com; font-src 'self'  
https://fonts.gstatic.com; img-src 'self' https://raw.githubusercontent.com data:;" always;  
  
    add_header Permissions-Policy "camera=(), microphone=(), geolocation=()" always;  
  
}
```

After adding these, validate with `nginx -t` and reload with `nginx -s reload`. The CSP policy should be tested to ensure it doesn't break the Ionicons CDN or inline styles.
Implementation time: ~15 minutes for an engineer with Nginx access.